

Kijani Battle Bot Platform Manual

Introduction

The **Kijani Project** is an open-source ESP32-based battle robot platform designed to make robot combat cheaper, simpler, and more accessible.

The platform combines:

- motor drivers
- battery charging
- WiFi control
- Bluetooth support
- web interfaces
- servo control
- custom firmware
- filesystem hosted web apps

all on a single compact PCB.

The goal is to allow anyone to:

- build a battle bot
- customise the controls
- experiment with robot designs
- create their own web interfaces
- learn embedded systems and robotics

Project Information

Item	Value
Project Name	Kijani
Original Author	Marc d'Hotman de Villiers
Main MCU	ESP32
Firmware Type	Arduino Framework
Control Methods	WiFi / Web / Bluetooth

Storage	LittleFS
Programming	USB Serial

History

The Kijani project started in 2024 after an interest in the antweight combat competitions run by the South African Robot League.

At the time:

- combat robots were expensive
- many separate electronics modules were required
- robot control systems were overly complicated

The original design goal was:

“Make a combat robot controller using a single ESP32 board.”

The earliest version used:

- Bluetooth control
- an Android app made using MIT App Inventor

In 2025 the project evolved further while assisting with the Africa Rover Moot project.

The design changed to:

- support all phones through web browsers
- simplify mass production
- reduce PCB size
- improve field usability

Version 2 of the PCB:

- reduced board length by 5mm
 - simplified the regulator system
 - reduced component count
 - improved manufacturability
-

Features

Main Features

- ESP32 based control system
 - Dual DC motor outputs
 - Two servo outputs
 - Single-cell LiPo charging
 - WiFi access point
 - Web-based control
 - LittleFS web hosting
 - USB programming
 - OTA-style web file uploads
 - Customisable HTML interfaces
 - Expandable API system
-

Hardware Overview

Main Components

Component	Purpose
ESP32	Main processor
CH340	USB serial programming
Motor Drivers	Dual DC motor control
Boost Regulator	Selectable higher voltage
LiPo Charger	Single-cell battery charging
Servo Outputs	Servo control
LittleFS	Web interface storage

Supported Robot Configurations

The platform is intentionally flexible.

Common Configurations

Standard Flipper Bot

- 2 drive motors
- 1 servo flipper

Spinner Bot

- 1 drive motor pair
- 1 spinning weapon motor

Steering Robot

- 1 drive motor
- 1 steering servo

Experimental Robots

- walking robots
 - lifting arms
 - autonomous projects
 - line followers
 - educational robots
-

Recommended Motors

Drive Motors

Recommended:

- N20 gear motors
- Micro metal gear motors
- TT motors (lightweight robots only)

Typical Voltage

- 3V–6V

Recommended Current

- under 1A continuous per motor

Important

Stall current can be much higher than running current.

Always test motors before combat use.

Recommended Servos

Recommended:

- SG90
- MG90S
- Metal gear micro servos

Servo Voltage Warning

The boost converter can raise voltage higher than some servos can tolerate.

Always verify:

- servo voltage rating
- regulator voltage settings
- battery voltage

before enabling boost mode.

Incorrect voltage can permanently damage servos.

Battery Support

Supported Batteries

Recommended

- 1S LiPo (3.7V nominal)

Typical Sizes

- 300mAh
 - 500mAh
 - 850mAh
 - 1000mAh
-

Battery Safety

IMPORTANT

LiPo batteries can be dangerous if:

- punctured
- overcharged
- short circuited
- physically damaged

Always:

- supervise charging
 - use correct polarity
 - inspect batteries regularly
 - stop using swollen batteries
-

PCB Information

The PCB integrates nearly everything needed for a battle robot.

Included Systems

- ESP32 module
- USB programming
- motor drivers
- battery charger
- boost regulator
- servo outputs
- filesystem support

PCB files are available through:

- EasyEDA
 - the hardware project folder
-

Firmware Overview

The robot software has two parts.

1. ESP32 Firmware

Written in:

- C++
- Arduino Framework

Responsibilities:

- motor control
- web server
- WiFi
- settings storage
- battery monitoring
- filesystem management

Programming is done through:

- USB serial
 - VSCode + PlatformIO
-

2. Web Interface Files

Stored in:

- LittleFS filesystem

Written in:

- HTML
- CSS
- JavaScript

These files:

- create the control interface
- display telemetry
- manage settings
- communicate with the API

The web interface can be fully customised.

Setup Guide

First Boot

After flashing the firmware:

1. Power on the board
 2. Wait for the WiFi access point
 3. Connect using a phone or computer
 4. Open the robot webpage
-

Default Access Point

Document:

- default SSID
- default password

here once finalised.

Changing Settings

Settings can be modified:

- from the web interface
- through the API

Examples:

- robot name
 - WiFi password
 - control settings
 - custom configuration values
-

Updating Web Files

Web files can be uploaded:

- through USB
- through the built-in web uploader

This allows rapid interface development without reflashing firmware.

Factory Reset

If the password is forgotten:

1. Power off the board
2. Short the PGM pins
3. Power on the board
4. Listen for the factory reset tune

The board will:

- erase stored settings
 - restore defaults
-

Warning Notes

Voltage Warning

The boost regulator can output voltages unsafe for:

- some servos
- low-voltage electronics

Always verify voltage compatibility before enabling higher voltages.

Motor Driver Warning

Large motors can exceed driver limits.

Avoid:

- stalled motors
- oversized drive motors
- direct short circuits

Cooling Warning

High current operation may generate heat.

Monitor:

- motor drivers
- regulators
- battery temperature

during testing.

Web API

The robot includes a built-in HTTP API.

This allows:

- custom control interfaces
- phone apps
- automation
- alternative control systems

Examples:

- `/processcontrol`
- `/quickstatus`
- `/files`
- `/updatesettings`

See the API Guide for full details.

File System

The ESP32 hosts files directly from LittleFS.

Examples:

- `/index.html`

- `/control.html`
- `/style.css`

This allows the robot itself to host its own web control pages.

Development Tools

Recommended software:

Tool	Purpose
VSCode	Development
PlatformIO	Firmware build/upload
LittleFS Upload	Web file upload
EasyEDA	PCB editing

Future Ideas

Potential future improvements:

- WebSocket control
- OTA firmware updates
- autonomous modes
- FPV support
- telemetry logging
- match replay
- camera streaming
- multiple weapon support